

LAPORAN PRAKTIKUM STRUKTUR DATA

“Implementasi Struktur Data Queue Berbasis Array dan LinkedList pada Java”

disusun Oleh:

Muhammad Faiz An-Anri

NIM 2511532021

Dosen Pengampu: Dr. Wahyudi S.T M.T

Asisten Pratikum: Jovantri Immanuel Gulo



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

DAFTAR ISI

DAFTAR ISI.....	i
KATA PENGANTAR.....	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN	2
2.1 Program QueueArray	2
2.2 Program QueueArrayDriver.....	3
2.3 Program QueueLinkedList	4
2.4 Program ReverseData	5
2.5 Program IterasiQueue.....	5
BAB III TUGAS.....	7
3.1 Program AntrianLoket.....	7
BAB IV KESIMPULAN	11
4.1 Ringkasan Hasil Praktikum.....	11
DAFTAR PUSTAKA	iii

KATA PENGANTAR

Ucapan syukur kita lafalkan kepada Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata kuliah Struktur Data.

Ucapan terima kasih disampaikan kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran konsep perulangan dalam bahasa pemrograman Java.

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Di dalam pengelolaan struktur data, urutan masuknya elemen sering kali menentukan urutan pemrosesannya. Prinsip ini dikenal sebagai First In, First Out (FIFO), yaitu data yang pertama kali masuk akan menjadi data yang pertama kali diproses. Struktur data yang menerapkan mekanisme ini disebut Queue (Antrean).

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar struktur data Queue beserta prinsip operasi utamanya yaitu First In, First Out (FIFO).
2. Mengimplementasikan operasi fundamental Queue secara manual menggunakan Array, meliputi enqueue, dequeue, front, dan rear.
3. Menerapkan antarmuka Queue menggunakan LinkedList serta manipulasi data tingkat lanjut, seperti iterasi elemen dan pembalikan urutan data (reverse) menggunakan bantuan struktur data Stack.

BAB 2

PEMBAHASAN

3.1 Program QueueArray

Program QueueArray adalah implementasi Queue berbasis array dengan memori statis. Ketika elemen belakang (rear) telah mencapai batas maksimum kapasitas array, posisinya dapat kembali ke indeks awal (0) selama masih terdapat ruang kosong yang ditinggalkan oleh operasi dequeue. Program ini juga dilengkapi dengan validasi isFull dan isEmpty untuk mencegah terjadinya error.

```

1 package pekan4_2511532021;
2 import java.util.*;
3
4 public class QueueArray_2511532021 {
5     int front_2021, rear_2021, size_2021;
6     int capacity_2021;
7     int array_2021[];
8
9     public QueueArray_2511532021 (int capacity){
10         this.capacity_2021 = capacity;
11         front_2021 = this.size_2021 = 0;
12         rear_2021 = capacity - 1;
13         array_2021 = new int[this.capacity_2021];
14     }
15
16     boolean isFull(QueueArray_2511532021 queue) {
17         return (queue.size_2021 == queue.capacity_2021);
18     }
19
20     boolean isEmpty(QueueArray_2511532021 queue) {
21         return (queue.size_2021 == 0);
22     }
23
24     void enqueue(int item) {
25         if (isFull(this)) {
26             return;
27         }
28
29         this.rear_2021 = (this.rear_2021 + 1) % this.capacity_2021;
30         this.array_2021[this.rear_2021] = item;
31         this.size_2021 = this.size_2021 + 1;
32         System.out.println(item + " enqueue to queue");
33     }
34
35     int dequeue() {
36         if (isEmpty(this)) {
37             return Integer.MIN_VALUE;
38         }
39
40         int item_2021 = this.array_2021[this.front_2021];
41         this.front_2021 = (this.front_2021 + 1) % this.capacity_2021;
42         this.size_2021 = this.size_2021 - 1;
43         return item_2021;
44     }
45
46     int front() {
47         if (isEmpty(this)) {
48             return Integer.MIN_VALUE;
49         }
50         return this.array_2021[this.front_2021];
51     }
52
53     int rear() {
54         if (isEmpty(this)) {
55             return Integer.MIN_VALUE;
56         }
57         return this.array_2021[this.rear_2021];
58     }
59
60     void display() {
61         int i_2021;
62         if (front_2021 == rear_2021) {
63             System.out.printf("\nAntrian Kosong\n");
64             return;
65         }
66
67         for (i_2021 = front_2021; i_2021 < rear_2021; i_2021++) {
68             System.out.printf("%d <--", array_2021[i_2021]);
69         }
70         return;
71     }
72 }
73
74

```

3.2 Program QueueArrayDriver

Program QueueArrayDriver berperan sebagai program utama (driver) untuk kelas QueueArray. Data dimasukkan secara berurutan (10, 20, 30, 40) menggunakan method enqueue. Ketika operasi dequeue dijalankan, nilai 10 menjadi data pertama yang dihapus dari struktur, membuktikan bahwa program berjalan sesuai dengan prinsip FIFO. Pengujian fungsionalitas front, rear, dan display juga berhasil dilakukan dengan menampilkan posisi elemen terdepan dan terakhir secara tepat.

```

1 package pekan4_2511532021;
2
3 public class QueueArrayDriver_2511532021 {
4     public static void main(String[] args) {
5         QueueArray_2511532021 queue_2021 = new QueueArray_2511532021(1000);
6         queue_2021.enqueue(10);
7         queue_2021.enqueue(20);
8         queue_2021.enqueue(30);
9         queue_2021.enqueue(40);
10        System.out.println("Item di depan " + queue_2021.front());
11        System.out.println("Item di depan " + queue_2021.rear());
12        System.out.println("Tampilan queue");
13        queue_2021.display();
14        System.out.println();
15        System.out.println(queue_2021.dequeue() + " dihapus dari queue");
16        System.out.println("Item di depan " + queue_2021.front());
17        System.out.println("Item di depan " + queue_2021.rear());
18        System.out.println("Tampilan queue");
19        queue_2021.display();
20    }
21 }

```

3.3 Program QueueLinkedList

Program QueueLinkedList memanfaatkan Java Collections Framework. Karena Queue di Java merupakan sebuah interface, instansiasinya dilakukan melalui kelas LinkedList. Dengan pendekatan ini, alokasi memori bersifat dinamis sehingga tidak perlu mendefinisikan kapasitas maksimum terlebih dahulu. Program menggunakan method bawaan seperti add() untuk operasi enqueue, remove() untuk operasi dequeue, serta peek() untuk melihat elemen terdepan tanpa menghapusnya dari antrean.

```

1 package pekan4_2511532021;
2 import java.util.*;
3
4 public class QueueLinkedList_2511532021 {
5     public static void main(String[] args) {
6         Queue<Integer> q_2021 = new LinkedList<>();
7         for (int i = 0 ; i < 6; i++) {
8             q_2021.add(i);
9         }
10
11         System.out.println("Elemen Antrian " + q_2021);
12
13         int hapus_2021 = q_2021.remove();
14
15         System.out.println("Hapus elemen = " + hapus_2021);
16         System.out.println(q_2021);
17
18         int depan_2021 = q_2021.peek();
19         System.out.println("Kepala Antrian = " + depan_2021);
20
21         int banyak_2021 = q_2021.size();
22         System.out.println("Size Antrian = " + banyak_2021);
23     }
24 }
25

```

3.4 Program ReverseData

Program ReverseData mendemonstrasikan implementasi algoritma yang menggabungkan karakteristik Queue (FIFO) dan Stack (LIFO) dengan tujuan membalikkan urutan data (1, 2, 3 menjadi 3, 2, 1). Logika program dijalankan dalam dua tahap perulangan. Pada tahap pertama, seluruh data dalam antrean di-remove lalu di-push ke dalam Stack. Pada tahap kedua, data dalam Stack di-pop dan di-add kembali ke dalam antrean. Melalui dua tahap tersebut, urutan data dalam antrean berhasil dibalik sepenuhnya.

```

1 package pekan4_2511532021;
2 import java.util.*;
3
4 public class ReverseData_2511532021 {
5     public static void main(String[] args) {
6         Queue<Integer> q_2021 = new LinkedList<Integer>();
7         q_2021.add(1);
8         q_2021.add(2);
9         q_2021.add(3);
10        System.out.println("sebelum reverse = " + q_2021);
11
12        Stack<Integer> s_2021 = new Stack<Integer>();
13        while (!q_2021.isEmpty()) {
14            s_2021.push(q_2021.remove());
15        }
16        while (!s_2021.isEmpty()) {
17            q_2021.add(s_2021.pop());
18        }
19        System.out.println("sesudah reverse = " + q_2021);
20    }
21 }
22
23

```

3.5 Program IterasiQueue

Program IterasiQueue mendemonstrasikan cara menelusuri elemen-elemen dalam antrian tanpa mengubah maupun menghapus data aslinya. Program menggunakan Iterator bertipe String, di mana perulangan while dikombinasikan dengan method hasNext() untuk memeriksa ketersediaan elemen berikutnya, dan method next() untuk menampilkan elemen tersebut ke layar. Penggunaan iterator membuat proses pembacaan data antrian menjadi lebih aman dan efisien, mulai dari elemen terdepan hingga elemen terbelakang.

```

1 package pekan4_2511532021;
2 import java.util.*;
3
4 public class IterasiQueue_2511532021 {
5     public static void main(String[] args) {
6         Queue<String> q_2021 = new LinkedList<String>();
7
8         q_2021.add("Praktikum");
9         q_2021.add("Struktur");
10        q_2021.add("data");
11        q_2021.add("dan");
12        q_2021.add("Algoritma");
13        Iterator<String> iterator = q_2021.iterator();
14        while (iterator.hasNext()) {
15            System.out.print(iterator.next() + " ");
16        }
17    }
18 }
19

```

BAB 3

TUGAS

3.6 Program AntrianLoket

Program AntrianLoket_2511532021 merupakan implementasi struktur data Queue berbasis array statis yang dirancang untuk mensimulasikan sistem antrian loket. Kelas ini memiliki empat atribut utama, yaitu front dan rear untuk menandai posisi elemen terdepan dan terbelakang, max untuk menyimpan kapasitas maksimum, serta array queue untuk menampung data berupa nama pelanggan. Konstruktor menginisialisasi front dan rear dengan nilai -1 sebagai penanda bahwa antrian masih kosong.

Kelas ini dilengkapi dengan empat method inti. Method isEmpty() dan isFull() berfungsi sebagai validasi kondisi antrian sebelum operasi dijalankan. Method enqueue() menambahkan nama pelanggan ke posisi rear selama antrian belum penuh, sementara method dequeue() menghapus elemen di posisi front dan menampilkan nama pelanggan yang telah dilayani. Method display() menelusuri elemen dari front hingga rear untuk menampilkan seluruh isi antrian, sedangkan method Reverse() membalik urutan elemen menggunakan teknik two-pointer lalu langsung memanggil display() untuk menampilkan hasilnya.

Method main() berperan sebagai program pengendali yang menyediakan antarmuka menu berbasis teks dengan lima pilihan operasi, yaitu tambah antrian, hapus antrian, tampilkan antrian, reverse, dan keluar. Program berjalan dalam perulangan do-while yang terus aktif selama pengguna belum memilih opsi keluar. Input pengguna dibaca menggunakan Scanner, dengan pemanggilan sc.nextLine() setelah sc.nextInt() untuk membersihkan buffer agar pembacaan nama pelanggan tidak terganggu.

```

1 package pekan4_2511532021;
2 import java.util.Scanner;
3
4 public class AntrianLoket_2511532021 {
5     int front;
6     int rear;
7     int max;
8     String[] queue;
9
10    public AntrianLoket_2511532021(int kapasitas) {
11        this.max = kapasitas;
12        this.queue = new String[max];
13        this.front = -1;
14        this.rear = -1;
15    }
16
17    boolean isEmpty() {
18        return front == -1;
19    }
20
21    boolean isFull() {
22        return rear == max - 1;
23    }
24
25    void enqueue(String nama) {
26        if (isFull()) {
27            System.out.println("Antrian penuh");
28            return;
29        }
30        if (isEmpty()) {
31            front = 0;
32        }
33        queue[++rear] = nama;
34        System.out.println("Data berhasil ditambahkan ke antrian");
35    }
36
37    void dequeue() {
38        if (isEmpty()) {
39            System.out.println("Antrian kosong");
40            return;
41        }
42        String nama = queue[front];
43        if (front == rear) {
44            front = -1;
45            rear = -1;
46        } else {
47            front++;
48        }
49        System.out.println(nama + " telah dilayani");
50    }
51
52    void display() {
53        if (isEmpty()) {
54            System.out.println("Antrian kosong");
55            return;
56        }
57        System.out.println("Isi antrian:");
58        int no = 1;
59        for (int i = front; i <= rear; i++) {
60            System.out.println(no + ". " + queue[i]);
61            no++;
62        }
63    }
64
65    void Reverse() {
66        if (isEmpty()) {
67            System.out.println("Antrian kosong");
68            return;
69        }
70        int i = front;
71        int j = rear;
72        while (i < j) {
73            String temp = queue[i];
74            queue[i] = queue[j];
75            queue[j] = temp;

```

```

73         String temp = queue[i];
74         queue[i] = queue[j];
75         queue[j] = temp;
76         i++;
77         j--;
78     }
79     display();
80 }
81
82 public static void main(String[] args) {
83     Scanner sc = new Scanner(System.in);
84     AntrianLoket_2511532021 loket = new AntrianLoket_2511532021(10);
85     int pilihan;
86
87     do {
88         System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
89         System.out.println("1. Tambah Antrian");
90         System.out.println("2. Hapus Antrian");
91         System.out.println("3. Tampilkan Antrian");
92         System.out.println("4. Reverse");
93         System.out.println("5. Keluar");
94         System.out.print("Pilih menu: ");
95
96         pilihan = sc.nextInt();
97         sc.nextLine();
98
99         switch (pilihan) {
100             case 1:
101                 System.out.print("Masukkan nama pelanggan: ");
102                 String nama = sc.nextLine();
103                 loket.enqueue(nama);
104                 break;
105             case 2:
106                 loket.dequeue();
107                 break;
108             case 3:
109                 loket.display();
110                 break;
111             case 4:
112                 loket.Reverse();
113                 break;
114             case 5:
115                 System.out.println("Program selesai");
116                 break;
117             default:
118                 System.out.println("Pilihan tidak valid");
119         }
120     } while (pilihan != 5);
121     sc.close();
122 }
123

```

BAB 4

KESIMPULAN

4.1 Ringkasan Hasil Praktikum

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Queue berbasis Array terbukti efektif dalam mensimulasikan sistem antrean nyata dengan menerapkan prinsip First In, First Out (FIFO). Alur masuk dan keluarnya data dikendalikan oleh dua penunjuk indeks, yakni front yang bertugas mengambil elemen terdepan melalui operasi dequeue, dan rear yang bertugas menyisipkan elemen baru di bagian belakang melalui operasi enqueue, keduanya dilengkapi dengan validasi isEmpty dan isFull untuk menjaga integritas data.

Selain itu, penggunaan memori statis berbasis Array juga menunjukkan fleksibilitas yang baik dalam hal manipulasi data. Hal ini dibuktikan melalui penerapan method Reverse yang berhasil membalikkan urutan elemen antrean menggunakan teknik swapping dari kedua ujung indeks secara bersamaan, tanpa memerlukan alokasi memori tambahan dalam prosesnya.